

Falsification de requête intersite Cross-Site Request Forgery – CSRF

Requêtes forcées, jetons, SameSite et validation de provenance

Comprendre les attaques, les contournements et la défense en profondeur

Module	Sécurité des applications web
Public	Débutant à praticien
Approche	Concepts, scénarios contrôlés, laboratoires et remédiation
Auteur	Gana Faye
Date	30 juillet 2026

À la fin de ce cours, l'apprenant doit pouvoir :

- ▶ définir précisément une CSRF et ses conditions d'exploitation ;
- ▶ distinguer envoi cross-site, lecture cross-origin et politique de même origine ;
- ▶ reconstruire une requête sensible dans un scénario de laboratoire ;
- ▶ analyser les jetons anti-CSRF et leurs erreurs de validation ;
- ▶ différencier un site, une origine et un domaine frère ;
- ▶ expliquer SameSite=Strict, Lax et None ;
- ▶ comprendre les contournements par méthode, gadget, domaine frère ou cookie récent ;
- ▶ auditer les contrôles fondés sur Origin et Referer ;
- ▶ concevoir une défense moderne, testable et redondante.

Autorisation

Les exemples sont réservés aux laboratoires, applications locales et environnements explicitement autorisés.

Données fictives

Les comptes, adresses et domaines utilisent des valeurs de démonstration.

Impact contrôlé

Les actions simulées concernent un changement d'adresse ou une préférence réversible.

But pédagogique

Comprendre les frontières du navigateur afin de concevoir et vérifier les défenses.

Ne jamais envoyer une preuve CSRF à un utilisateur ou service tiers

Parcours du module



1

Fondamentaux, fonctionnement et impact

Module de sécurité des applications web

Qu'est-ce qu'une CSRF ?

Une falsification de requête intersite est une vulnérabilité qui permet à un attaquant de pousser le navigateur d'une victime authentifiée à envoyer une requête non désirée vers une application cible.

Victime

Possède une session valide sur l'application cible.

Attaquant

Contrôle une page, un lien, un e-mail ou un contenu chargé par la victime.

Navigateur

Ajoute automatiquement certains cookies à la requête vers la cible.

L'application confond une requête authentifiée avec une intention authentique

Une CSRF exploitable réunit généralement :

1. une **action pertinente**, qui modifie des données ou un état métier ;
2. une **authentification fondée sur des cookies** automatiquement joints par le navigateur ;
3. des **paramètres prévisibles**, que l'attaquant peut reconstruire sans lire la réponse.

Exemples d'actions

Changer une adresse e-mail, modifier un mot de passe, ajouter un bénéficiaire, publier un contenu, changer une préférence ou lancer une opération administrative.

Authentification et intention : deux preuves différentes

Cookie de session

Prouve que la requête est rattachée à une session reconnue. Il ne prouve pas comment la navigation a été déclenchée.

Preuve anti-CSRF

Prouve que la requête provient d'un flux autorisé ou contient un secret inaccessible au site attaquant.

Qui est connecté ?

≠

Qui a voulu cette action ?

Quel est l'impact d'une CSRF ?

L'impact dépend entièrement des actions accessibles à la victime :

- ▶ modification du profil, de l'adresse ou des préférences ;
- ▶ changement d'adresse e-mail facilitant une prise de contrôle ultérieure ;
- ▶ ajout d'un compte, d'une clé, d'un bénéficiaire ou d'une règle ;
- ▶ publication ou suppression de contenu sous l'identité de la victime ;
- ▶ action administrative si la victime possède des privilèges élevés ;
- ▶ affaiblissement des paramètres de sécurité ;
- ▶ déclenchement d'une transaction si aucun contrôle additionnel n'existe.

Ce que la CSRF ne permet pas nécessairement

- ▶ lire librement la réponse de l'application cible ;
- ▶ voler directement le cookie `HttpOnly` ;
- ▶ choisir des paramètres secrets inconnus ;
- ▶ contourner une confirmation forte ou une réauthentification ;
- ▶ produire un effet si les cookies ne sont pas envoyés ;
- ▶ franchir un contrôle d'autorisation métier déjà appliqué à l'utilisateur.

La CSRF force souvent une action sans pouvoir observer directement son résultat

Politique de même origine : rôle exact

La politique de même origine limite les interactions d'un script avec une origine différente. Elle protège fortement la **lecture** de nombreuses réponses, mais certaines requêtes cross-origin peuvent encore être **envoyées** par des formulaires, images ou navigations.

Opération	Effet général
envoyer un formulaire simple	souvent possible cross-origin
charger une image ou naviguer	généralement possible
lire la réponse avec JavaScript	bloqué sans autorisation CORS appropriée
accéder au DOM d'une autre origine	bloqué par la politique de même origine

Comment fonctionne une attaque CSRF ?



site attaquant n'a pas besoin de connaître le cookie : le navigateur décide de l'envoyer selon les règles applicables.

Scénario jouet : changement d'adresse e-mail

1. Alice est connectée à `https://account.example` ;
2. l'action légitime utilise `POST /profile/email` ;
3. le serveur ne vérifie que le cookie de session ;
4. Alice ouvre une page contrôlée sur `attacker.example` ;
5. la page soumet un formulaire vers l'application cible ;
6. le navigateur joint le cookie si la politique le permet ;
7. l'adresse est modifiée sans intention d'Alice.

Question de validation : Quel élément manque au serveur pour distinguer le formulaire légitime du formulaire forgé ?

CSRF, XSS et clickjacking

Attaque	Primitive	But principal
CSRF	navigateur envoie une requête authentifiée	forcer une action
XSS	script s'exécute dans l'origine cible	lire, modifier et agir dans la session
clickjacking	victime clique sur une interface masquée	détourner une interaction réelle

Une XSS peut généralement contourner les protections CSRF, car le script s'exécute dans l'origine autorisée et peut lire les jetons présents dans le DOM.

- ▶ la CSRF exploite la confiance accordée aux requêtes portant des cookies ;
- ▶ le navigateur peut envoyer une requête qu'un script attaquant ne peut pas lire ;
- ▶ l'impact dépend des fonctions de la victime ;
- ▶ la session prouve une identité, pas l'intention de l'action ;
- ▶ une défense robuste ajoute une preuve non reproductible par un site tiers.

2

Construire et analyser une exploitation CSRF

Module de sécurité des applications web

1. capturer une requête sensible légitime ;
2. identifier méthode, chemin, paramètres, cookies et en-têtes ;
3. déterminer les valeurs prévisibles ;
4. retirer ou modifier les contrôles anti-CSRF ;
5. vérifier quels formats cross-origin sont acceptés ;
6. reconstruire une preuve de concept locale ;
7. confirmer l'effet avec son propre compte de test ;
8. documenter la cause et retester après correction.

Requête légitime de référence

```
POST /profile/email HTTP/1.1
Host: account.example
Cookie: session=SESSION_ALICE
Content-Type: application/x-www-form-urlencoded

email=alice.new@example.test
```

Observation

La requête ne contient aucun jeton imprévisible, aucun en-tête personnalisé et aucune preuve de provenance validée.

Questions à poser sur la requête

- ▶ le navigateur ajoute-t-il automatiquement l'authentification ?
- ▶ l'action accepte-t-elle un formulaire HTML simple ?
- ▶ les paramètres sont-ils connus ou devinables ?
- ▶ le serveur exige-t-il un jeton lié à la session ?
- ▶ la méthode peut-elle être changée ?
- ▶ le paramètre du jeton peut-il être supprimé ?
- ▶ Origin, Referer ou les Fetch Metadata sont-ils validés ?
- ▶ les cookies SameSite seront-ils envoyés dans ce contexte ?

Preuve de concept HTML contrôlée

```
<form action="https://account.example/profile/email"
      method="POST" id="csrf-form">
  <input type="hidden" name="email"
        value="lab-user@example.test">
</form>
<script>
  document.getElementById('csrf-form').submit();
</script>
```

Cette page ne doit être utilisée que contre un compte et une cible de laboratoire autorisés.

Pourquoi un formulaire HTML est important

Un formulaire cross-origin peut envoyer des requêtes dites « simples » sans autorisation CORS préalable. Les formats courants incluent :

- ▶ `application/x-www-form-urlencoded`;
- ▶ `multipart/form-data`;
- ▶ `text/plain` dans certains scénarios;
- ▶ les navigations GET avec paramètres dans l'URL.

CORS n'est pas une protection générale contre les formulaires CSRF

Requête GET modifiant un état

Une action sensible ne doit pas utiliser GET. Les navigateurs et intermédiaires considèrent cette méthode comme sûre et peuvent la déclencher via :

- ▶ un lien ;
- ▶ une image ;
- ▶ une redirection ;
- ▶ un préchargement ;
- ▶ une navigation de premier niveau.

Anti-modèle

GET /account/delete?confirm=true transforme une simple navigation en opération destructive.

Dans un laboratoire, l'exploit peut être hébergé sur un serveur dédié puis ouvert par le compte de test. Dans un audit professionnel, la preuve doit rester contrôlée :

- ▶ cible et compte appartenant au périmètre ;
- ▶ action réversible et valeur clairement marquée ;
- ▶ absence d'envoi à de vrais utilisateurs ;
- ▶ journalisation de l'heure et de la requête ;
- ▶ restauration immédiate de l'état ;
- ▶ capture du résultat métier, pas seulement du statut HTTP.

Pourquoi un code 200 ne suffit pas

Une réponse de succès peut être une page d'erreur applicative ou une redirection vers la connexion. La validation doit vérifier :

- ▶ que le cookie de la victime a été inclus ;
- ▶ que le serveur a traité la bonne session ;
- ▶ que la donnée a réellement changé ;
- ▶ qu'aucune interaction supplémentaire n'était nécessaire ;
- ▶ que l'effet vient de la requête forgée et non d'une action manuelle.

Limites fréquentes d'une exploitation

- ▶ cookie SameSite non envoyé ;
- ▶ action exigeant un ancien mot de passe ;
- ▶ jeton anti-CSRF imprévisible ;
- ▶ paramètre inconnu ou spécifique à l'utilisateur ;
- ▶ contrôle d'origine correctement appliqué ;
- ▶ en-tête personnalisé imposant une prérequête CORS ;
- ▶ confirmation hors bande ou authentification multifacteur ;
- ▶ interaction utilisateur trop spécifique.

- ▶ partir d'une requête légitime, jamais d'une supposition ;
- ▶ ne modifier qu'un contrôle à la fois ;
- ▶ reproduire méthode, paramètres et format ;
- ▶ tenir compte du contexte SameSite ;
- ▶ prouver l'effet métier avec un compte autorisé ;
- ▶ restaurer les données après le test.

3

Jetons anti-CSRF et erreurs de validation

Module de sécurité des applications web

Qu'est-ce qu'un jeton CSRF ?

Un jeton anti-CSRF est une valeur secrète, imprévisible et validée côté serveur. L'application la fournit à la page légitime, puis exige qu'elle accompagne toute requête sensible.

Secret

Le site attaquant ne peut pas lire la page légitime pour récupérer la valeur.

Imprévisible

La valeur résiste à la devinette et possède suffisamment d'entropie.

Lié

Le serveur l'associe à la session, à l'utilisateur ou à une requête vérifiable.

Transmission dans un formulaire

```
<form action="/profile/email" method="POST">
  <input type="hidden" name="csrf"
        value="RANDOM_SESSION_BOUND_TOKEN">
  <input type="email" name="email">
  <button>Enregistrer</button>
</form>
```

Le serveur refuse la requête si la valeur est absente, invalide, expirée ou associée à une autre session.

Synchronizer token pattern

1. le serveur crée une session ;
2. il génère ou associe un jeton secret à cette session ;
3. la page légitime reçoit le jeton dans le HTML ou une réponse JSON ;
4. le navigateur le renvoie dans le corps ou un en-tête personnalisé ;
5. le serveur compare la valeur à celle attendue pour la session.

Propriété

Le site attaquant peut provoquer une requête, mais il ne peut normalement pas lire la page cible afin d'obtenir la valeur correcte.

Propriétés d'un bon jeton

- ▶ généré avec un générateur cryptographiquement sûr ;
- ▶ suffisamment long et imprévisible ;
- ▶ lié à la session ou authentifié cryptographiquement ;
- ▶ validé sur toutes les opérations sensibles ;
- ▶ comparé de manière sûre ;
- ▶ non placé dans une URL, un journal ou un référent ;
- ▶ renouvelé selon le modèle de session ;
- ▶ jamais accepté lorsqu'il est absent.

- ▶ contrôle appliqué uniquement à certaines méthodes ;
- ▶ contrôle exécuté seulement si le paramètre est présent ;
- ▶ jeton valide globalement mais non lié à la session ;
- ▶ jeton lié à un cookie secondaire contrôlable ;
- ▶ valeur simplement dupliquée dans un cookie ;
- ▶ comparaison partielle ou format prévisible ;
- ▶ fuite du jeton par URL, journal ou ressource tierce ;
- ▶ endpoint oublié ou ancienne version non protégée.

Validation dépendante de la méthode

Une application peut vérifier le jeton pour POST, mais accepter la même action en GET ou après surcharge de méthode.

Défaut

La politique est attachée à la méthode attendue, pas à l'opération métier.

Correction

Appliquer le contrôle à l'action elle-même et refuser toute méthode non explicitement autorisée.

Question de validation : Pourquoi changer POST en GET ne devrait-il jamais désactiver une vérification de sécurité ?

Validation dépendante de la présence du jeton

Certains traitements suivent une logique incorrecte :

```
si csrf est présent : vérifier ; sinon : continuer
```

Un attaquant supprime alors entièrement le paramètre. Une valeur vide et un paramètre absent doivent tous deux provoquer un refus pour une action protégée.

Absence de preuve = refus, jamais contournement

Jeton non lié à la session utilisateur

Si le serveur accepte n'importe quel jeton valide émis à n'importe quel utilisateur, l'attaquant peut :

1. ouvrir sa propre session ;
2. récupérer son jeton légitime ;
3. placer ce jeton dans la requête forgée ;
4. faire envoyer la requête avec le cookie de la victime ;
5. profiter de l'absence de liaison jeton-session.

Correction

Comparer le jeton à la valeur attendue pour la session authentifiée qui porte la requête.

Jeton lié à un cookie non sessionnel

Certaines applications lient le jeton à un cookie séparé, mais autorisent un sous-domaine ou une fonctionnalité à définir ce cookie.

1. l'attaquant obtient un couple valeur-cookie valide ;
2. il exploite une possibilité d'injection de cookie dans le domaine ;
3. le navigateur de la victime reçoit le cookie choisi ;
4. la requête forgée contient la valeur correspondante ;
5. la comparaison réussit sans relation avec la vraie session.

Pourquoi les domaines frères comptent

Un cookie défini avec `Domain=example.com` peut être envoyé à plusieurs sous-domaines. Une vulnérabilité sur `blog.example.com` peut donc affaiblir la défense de `account.example.com` si elle permet de fixer un cookie de domaine.

Réduction du risque

Préférer les cookies liés à l'hôte, utiliser le préfixe `__Host-`, omettre `Domain` et protéger tous les sous-domaines.

Jeton simplement dupliqué dans un cookie

Le modèle *double submit* naïf compare uniquement :

`cookie csrf = paramètre csrf`

Si l'attaquant peut fixer le cookie, il choisit la même valeur dans le formulaire. Le serveur observe une égalité mais aucune preuve de liaison avec la session.

Version robuste

Signer ou authentifier la valeur avec un secret serveur et lier le calcul à l'identité de session.

Jeton par session ou par requête ?

Modèle	Avantage	Contrainte
par session	simple et compatible avec plusieurs onglets	exposition plus longue en cas de fuite
par requête	réduit la fenêtre de réutilisation	navigation arrière, concurrence et ergonomie plus complexes
signé sans état	validation distribuée	conception cryptographique et liaison à la session indispensables

Le choix dépend de l'architecture ; la propriété indispensable reste la liaison vérifiable à la session concernée.

Tests systématiques d'un jeton

1. valeur correcte et session correcte ;
2. valeur absente ;
3. valeur vide ;
4. valeur aléatoire ;
5. jeton d'une autre session ;
6. jeton expiré ou déjà utilisé ;
7. méthode différente ;
8. endpoint alternatif ou ancien ;
9. cookie secondaire modifié ;
10. paramètre dupliqué ou encodé différemment.

- ▶ un jeton doit être secret, imprévisible et correctement lié ;
- ▶ la vérification doit être obligatoire sur chaque action sensible ;
- ▶ un jeton valide d'un autre compte ne doit jamais fonctionner ;
- ▶ les cookies secondaires et sous-domaines élargissent la frontière de confiance ;
- ▶ le double submit naïf ne suffit pas si le cookie peut être fixé.

4

Cookies SameSite et scénarios de contournement

Module de sécurité des applications web

Qu'est-ce qu'un « site » pour SameSite ?

Un site correspond généralement au schéma et au domaine enregistrable, souvent appelé *scheme + eTLD+1*.

`https` + `example.com`

Ainsi, `https://shop.example.com` et `https://account.example.com` sont généralement **same-site**, mais restent des origines différentes.

Quelle est la différence entre site et origine ?

Concept	Composition	Exemple de comparaison
origine	schéma + hôte + port	deux sous-domaines différents sont cross-origin
site	schéma + domaine enregistable	ces deux hôtes HTTPS sont same-site

Conséquence

Un domaine frère compromis peut produire des requêtes same-site, même s'il ne partage pas la même origine.

Exemples site / origine

URL A	URL B	Site	Origine
https://a.example.com	https://b.example.com	même	différente
https://example.com	https://example.com:8443	même	différente
http://example.com	https://example.com	différente	différente
https://example.com	https://example.net	différente	différente

Comment fonctionne SameSite ?

L'attribut SameSite indique au navigateur dans quels contextes cross-site un cookie peut accompagner une requête. Il réduit l'envoi automatique des cookies, mais ne remplace pas toutes les autres défenses.

Strict

Cookie réservé aux requêtes same-site.

Lax

Autorise notamment certaines navigations de premier niveau avec méthode sûre.

None

Autorise le cross-site ; exige normalement Secure.

- ▶ offre la restriction cross-site la plus forte ;
- ▶ peut gêner les liens entrants et certains parcours fédérés ;
- ▶ n'empêche pas un scénario déclenché depuis un domaine frère same-site ;
- ▶ peut être contourné par un gadget de redirection côté client situé sur le site ;
- ▶ ne corrige pas une XSS dans l'origine cible.

- ▶ bloque de nombreuses requêtes cross-site non sûres ;
- ▶ peut envoyer le cookie lors d'une navigation de premier niveau en GET ;
- ▶ protège moins bien les actions accessibles par GET ;
- ▶ peut être affecté par une surcharge de méthode ;
- ▶ certains navigateurs appliquent un comportement Lax par défaut lorsque l'attribut est absent.

- ▶ permet l'envoi du cookie dans des contextes cross-site ;
- ▶ est nécessaire pour certains widgets, intégrations et authentifications fédérées ;
- ▶ doit être associé à Secure dans les navigateurs modernes ;
- ▶ laisse l'application dépendre fortement des jetons et contrôles de provenance ;
- ▶ doit être réservé aux cookies qui nécessitent réellement ce comportement.

Contourner Lax avec une requête GET

Si une action sensible accepte GET, une navigation de premier niveau peut inclure le cookie Lax. Le problème est alors double :

1. l'action utilise une méthode considérée comme sûre ;
2. la politique Lax autorise précisément certaines navigations sûres.

Correction

Réserver GET à la consultation et protéger toute mutation avec une méthode non sûre et un jeton valide.

Certaines applications acceptent une requête GET contenant un paramètre comme `_method=POST`. Le navigateur évalue SameSite selon la méthode HTTP réelle, tandis que le framework traite ensuite la requête comme une mutation.

Désalignement

Le navigateur voit une navigation GET autorisée par Lax; l'application voit une opération POST après réécriture.

Méthodologie conceptuelle du scénario PortSwigger :

1. observer une fonction de changement d'e-mail en POST ;
2. confirmer qu'une surcharge de méthode est acceptée ;
3. transformer l'appel en navigation GET ;
4. conserver les paramètres de l'action ;
5. vérifier que le cookie Lax accompagne la navigation ;
6. confirmer le changement avec le compte de laboratoire.

Contourner SameSite avec un gadget sur site

Un gadget sur site est une fonctionnalité légitime capable de provoquer une navigation ou une requête vers une destination influençable. Exemples :

- ▶ redirection côté client fondée sur un paramètre ;
- ▶ route de retour après connexion ;
- ▶ lien de prévisualisation ;
- ▶ fonction de partage ou de navigation interne.

Une fois la navigation relayée par le site cible, la requête secondaire peut être considérée comme same-site.

Chaîne d'un gadget de redirection



Les sous-domaines d'un même domaine enregistrable sont souvent same-site. Une XSS ou une redirection ouverte sur `blog.example.com` peut donc affaiblir les cookies de `account.example.com`.

Erreur

Évaluer uniquement la sécurité du sous-domaine qui traite l'action.

Approche

Inventorier tous les domaines frères, leurs propriétaires, leurs cookies et leurs gadgets.

Pour préserver certains parcours de connexion, des navigateurs peuvent appliquer temporairement un comportement plus permissif à un cookie récemment créé sans attribut SameSite explicite. Une attaque peut chercher à :

1. provoquer un renouvellement de session ou un parcours OAuth ;
2. exploiter la courte fenêtre suivant l'émission ;
3. déclencher rapidement une requête cross-site non sûre.

- ▶ un bouton ou une fenêtre ouvre le fournisseur d'identité ;
- ▶ le retour OAuth émet un nouveau cookie de session ;
- ▶ la page attaquante conserve une référence à la fenêtre ou déclenche ensuite la navigation ;
- ▶ l'action CSRF est tentée pendant la fenêtre de compatibilité ;
- ▶ le succès dépend du navigateur et de la configuration exacte du cookie.

Correction

Définir explicitement SameSite, utiliser un état OAuth robuste et conserver une protection anti-CSRF sur les actions métier.

SameSite : défense utile mais partielle

- ▶ dépend du contexte de navigation et du navigateur ;
- ▶ raisonne en termes de site, pas d'origine ;
- ▶ peut être affaibli par des gadgets ou domaines frères ;
- ▶ ne protège pas si l'action sensible est un GET ;
- ▶ ne remplace pas un jeton pour les opérations critiques ;
- ▶ ne corrige ni XSS ni défaut d'autorisation.

- ▶ un site et une origine sont deux frontières différentes ;
- ▶ Strict est plus restrictif, Lax autorise certaines navigations GET ;
- ▶ None nécessite une défense anti-CSRF indépendante ;
- ▶ méthodes surchargées, gadgets et domaines frères créent des contournements ;
- ▶ les cookies doivent déclarer explicitement la politique attendue.

5

Défenses fondées sur Origin et Referer

Module de sécurité des applications web

- ▶ Origin indique généralement le schéma, l'hôte et le port à l'origine de la requête ;
- ▶ Referer peut contenir l'URL de la ressource précédente, selon la politique de référent ;
- ▶ ces en-têtes sont utiles comme couche de défense ;
- ▶ ils doivent être comparés par analyse structurée, jamais par simple recherche de chaîne ;
- ▶ la politique doit définir le comportement lorsque l'en-tête est absent ou null.

Validation dépendante de la présence du Referer

Une logique vulnérable peut accepter la requête lorsque Referer est absent :

`si Referer existe : vérifier ; sinon : autoriser`

Une page peut influencer l'envoi du référent au moyen d'une politique adaptée. L'absence de provenance ne doit donc pas devenir un chemin de contournement silencieux.

Referer-Policy et suppression du référent

La politique de référent contrôle la quantité d'informations envoyée. Dans un laboratoire, une page peut demander à ne transmettre aucun référent.

Défaut serveur

Si la vérification n'est exécutée que lorsque l'en-tête existe, sa suppression transforme une protection en mécanisme optionnel.

Correction

Refuser les requêtes sensibles dont la provenance ne peut pas être établie, sauf flux explicitement documenté.

Une comparaison comme `Referer` contient `example.com` peut accepter :

- ▶ `https://example.com.attacker.test/;`
- ▶ `https://attacker.test/?next=example.com;`
- ▶ un nom d'utilisateur ou fragment contenant la chaîne attendue ;
- ▶ une représentation ambiguë mal analysée.

Analyser l'URL, extraire l'origine, puis comparer exactement

Origin ou Referer ?

En-tête	Avantage	Limite
Origin	valeur réduite et structurée	absent dans certains contextes, parfois null
Referer	disponible dans de nombreux flux	dépend de Referrer-Policy et contient davantage d'informations

Une stratégie courante vérifie `Origin` lorsqu'il est présent, puis utilise prudemment `Referer` comme solution secondaire.

Derrière un proxy inverse, l'application doit connaître son origine publique réelle. Une mauvaise confiance dans Host ou X-Forwarded-Host peut fausser la comparaison.

Bonne pratique

Configurer explicitement les origines autorisées et ne faire confiance aux en-têtes de proxy que lorsqu'ils proviennent d'intermédiaires connus.

Tests des défenses de provenance

1. origine légitime exacte ;
2. origine externe ;
3. sous-domaine non autorisé ;
4. origine attendue dans une sous-chaîne ;
5. en-tête absent ;
6. valeur `null` ;
7. schéma différent ;
8. port différent ;
9. en-têtes de proxy manipulés ;
10. requête same-site mais cross-origin.

- ▶ comparer des origines structurées et exactes ;
- ▶ définir une politique de refus pour les en-têtes absents ;
- ▶ ne pas utiliser une simple sous-chaîne ;
- ▶ tenir compte des proxies et de l'origine publique ;
- ▶ utiliser la provenance comme couche complémentaire, pas comme unique défense.

6

Prévention et défense en profondeur

Module de sécurité des applications web

Une action sensible doit exiger une preuve qu'un site tiers ne peut pas reproduire, même s'il peut pousser le navigateur à envoyer une requête.

session valide

+

jeton ou provenance valide

+

autorisation métier

1. ne jamais modifier l'état via GET ;
2. utiliser la protection CSRF du framework ;
3. valider un jeton lié à la session ;
4. définir explicitement SameSite ;
5. vérifier Origin/Referer ou Fetch Metadata ;
6. exiger une réauthentification pour les opérations critiques ;
7. protéger les sous-domaines et la configuration des cookies ;
8. tester tous les endpoints et méthodes alternatives.

Les frameworks modernes proposent généralement :

- ▶ génération et validation de jetons ;
- ▶ intégration aux formulaires ;
- ▶ middleware pour les méthodes non sûres ;
- ▶ rotation et liaison à la session ;
- ▶ mécanismes adaptés aux requêtes AJAX.

Attention

Désactiver globalement la protection pour résoudre un problème d'intégration crée souvent des endpoints oubliés et non protégés.

Une application monopage peut transmettre le jeton dans un en-tête comme `X-CSRF-Token`. Un formulaire HTML cross-origin ne peut pas librement ajouter cet en-tête, et une requête JavaScript cross-origin doit satisfaire CORS.

Conditions

Le jeton doit rester lié à la session, CORS doit être strict et les origines autorisées ne doivent jamais refléter arbitrairement l'entrée.

Les en-têtes `Sec-Fetch-Site`, `Sec-Fetch-Mode`, `Sec-Fetch-Dest` et `Sec-Fetch-User` aident le serveur à comprendre le contexte de navigation. Une politique peut refuser les mutations lorsque `Sec-Fetch-Site: cross-site`, tout en prévoyant des exceptions documentées pour les clients qui ne transmettent pas ces en-têtes.

Exemple de cookie de session fortement lié à l'hôte :

```
Set-Cookie: __Host-session=...; Path=/; Secure; HttpOnly;  
           SameSite=Lax
```

- ▶ Secure limite le transport à HTTPS ;
- ▶ HttpOnly limite l'accès JavaScript au cookie ;
- ▶ absence de Domain limite le cookie à l'hôte ;
- ▶ Path=/ est requis par le préfixe __Host- ;
- ▶ SameSite réduit les contextes cross-site.

Pour une action critique, une confirmation explicite réduit l'impact d'une CSRF :

- ▶ demander le mot de passe actuel ;
- ▶ exiger un facteur supplémentaire ;
- ▶ afficher clairement la cible et l'effet ;
- ▶ utiliser un challenge à durée courte ;
- ▶ lier la confirmation à l'action exacte ;
- ▶ invalider le challenge après utilisation.

Une API authentifiée uniquement par un en-tête `Authorization: Bearer ...` que le navigateur n'ajoute pas automatiquement est généralement moins exposée à la CSRF classique.

Nuance

Si le jeton est stocké dans un cookie, si l'application accepte aussi une session cookie ou si CORS est trop permissif, le risque réapparaît. Le stockage côté navigateur introduit également d'autres risques, notamment XSS.

CORS n'est pas un jeton CSRF

- ▶ CORS contrôle surtout la lecture des réponses par JavaScript ;
- ▶ les formulaires simples ne nécessitent pas toujours de prérequête ;
- ▶ une mauvaise configuration avec identifiants peut aggraver l'exposition ;
- ▶ autoriser arbitrairement des origines détruit la frontière attendue ;
- ▶ la protection CSRF doit rester explicite pour les endpoints à cookies.

Matrice de contrôles

Risque	Contrôle principal	Contrôles complémentaires
formulaire cross-site action via GET cookie fixé par sous-domaine opération critique API AJAX réfèrent absent	jeton lié à la session méthode correcte cookie hôte / __Host- réauthentification en-tête personnalisé politique de refus	SameSite, Origin, Fetch Metadata jeton et refus des surcharges hygiène des sous-domaines confirmation et audit CORS strict et jeton Origin et jeton

Checklist de développement

- ☐ aucune mutation en GET ;
- ☐ middleware CSRF actif ;
- ☐ jeton lié à la session ;
- ☐ absence = refus ;
- ☐ toutes les méthodes couvertes ;
- ☐ SameSite explicite ;
- ☐ cookies liés à l'hôte ;
- ☐ Origin/Referer structurés ;
- ☐ Fetch Metadata évalué ;
- ☐ CORS strict ;
- ☐ confirmation des actions critiques ;
- ☐ tests négatifs automatisés.

Plan de tests de non-régression

1. requête légitime avec jeton correct ;
2. jeton absent, vide, aléatoire et d'une autre session ;
3. changement de méthode et surcharge ;
4. tentative de formulaire cross-site ;
5. navigation GET de premier niveau ;
6. origine externe, sous-domaine et en-tête absent ;
7. cookies Strict, Lax et None selon les parcours ;
8. gadget de redirection et domaine frère ;
9. endpoint historique ou version API alternative ;
10. confirmation que les refus ne modifient aucune donnée.

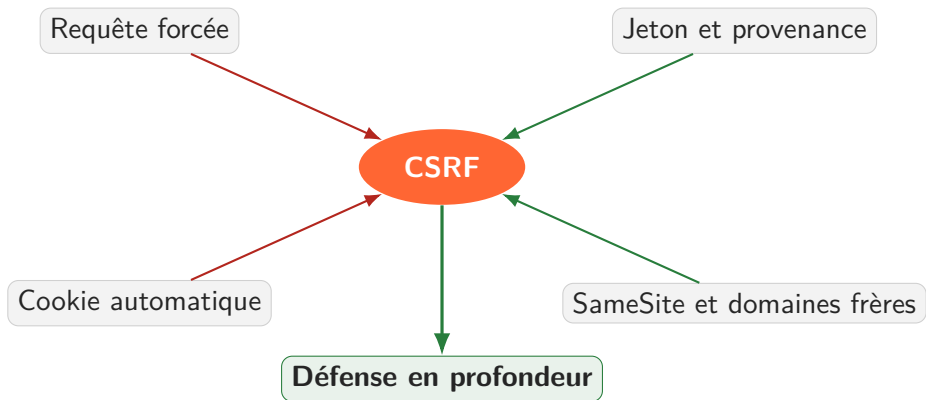
Architecture cible sécurisée



Une application utilise SameSite=Lax, protège POST /email par un jeton, mais accepte aussi GET /email?_method=POST. Le contrôle du jeton n'est exécuté que pour la méthode HTTP POST réelle.

Question de validation : Identifiez le désalignement entre navigateur et framework, expliquez pourquoi le cookie peut être envoyé et proposez trois corrections indépendantes.

Carte mentale finale



- ▶ PortSwigger Web Security Academy, *Cross-site request forgery*.
- ▶ PortSwigger, *Bypassing CSRF token validation*.
- ▶ PortSwigger, *Bypassing SameSite cookie restrictions*.
- ▶ OWASP Cheat Sheet Series, *CSRF Prevention Cheat Sheet*.
- ▶ MDN Web Docs, *Cross-site request forgery*.
- ▶ MDN Web Docs, *Set-Cookie*.

Fin du cours

La session authentifiée. Le contrôle anti-CSRF confirme l'intention.

Jeton lié, cookies maîtrisés, provenance vérifiée et actions sensibles confirmées.

Scénarios exclusivement réservés aux environnements autorisés.